Localization and Navigation for Indoor Mobile Robot Based on ROS

Yong Li
School of Automation
Chongqing University of Posts and Telecommunications
Chongqing, China
023liyong@163.com

Changxing Shi
School of Automation
Chongqing University of Posts and Telecommunications
Chongqing, China
schangxing@163.com

***Abstract*—In view of the high cost and complex structure of autonomous navigation robots in indoor environment, a low-cost, high-performance and simple structure scheme of mobile robot positioning and navigation is proposed and verified experimentally. A modular mobile robot platform was designed and the robot control system was divided into two parts: the upper control system and the real-time control system, which improved the stability of the system and reduced the coupling between modules. Lidar and RGB-D camera Kinect are used as main sensors to obtain information about the surrounding environment. The robot software system is developed by using a distributed software framework of robot operating system (ROS). The SLAM algorithm based on particle filter is used to achieve Simultaneous Localization and Mapping (SLAM) in an unknown environment. The experimental results show that the system can build a map that is consistent with the environment in the indoor environment, and can complete the autonomous navigation task according to the created map. It not only has low cost and high performance, but also has the characteristics of short development cycle and easy expansion.**

Keywords—simultaneous localization and mapping, mobile robot, particle filtering, robot operating system, navigation

I. INTRODUCTION

With the rapid development of robotics technology, indoor service mobile robots have become more and more common in life, but there are still many problems in indoor mobile robots. The first problem to be solved is to achieve autonomous navigation in indoor environments. However, in the indoor environment, the GPS signal is weak, and the civilian GPS positioning accuracy cannot reach the centimeter-level accuracy required for autonomous navigation of the robot. To achieve autonomous navigation indoors, mobile robots must be able to use their own sensors to obtain environmental information, construct a map that is consistent with their environment, and use maps to determine their location, which is called Simultaneous Localization and Mapping (SLAM) [1]。SLAM technology has become the focus of many research institutions and technology companies, and has been used to developed a variety of intelligent mobile robots, such as personal robot PR2 [2] developed by Willow Garage, service robot Pepper [3] developed by Japan Soft Bank, PowerBot and Pioneer series robots [4] developed by Activmedia, robot platform Husky [5] developed by Clearpath Robotics, indoor autonomous heel robot CASIA-I developed by CAS, and multi-functional intelligent mobile robot experiment platform THMR-V [6] developed by Tsinghua University. Although these mobile robots have powerful performance and complete software and hardware support, but they are very expensive and have poor system scalability. Moreover, most of the robots' software codes are not open source and have poor portability and reusability, which is not conducive to secondary development. However, developing a mobile robot hardware and software system from scratch is very difficult and it takes a long time.

This work is supported by Chongqing Commission of Urban and Rural Development Project No.2018-1-1-7

The main task of this research is to use the open source software platform robot operating system (ROS) as the core to design a set of mobile robot systems with low cost, high performance and scalability [7]. In this system, SLAM, navigation and obstacle avoidance were studied, and the functions of map building and autonomous navigation of mobile robots were realized.

II. SLAM BASED ON PARTICLE FILTER

When the mobile robot is performing synchronous positioning and map building, the main problem to be solved is how to use the sensor information to estimate its own pose. The motion equations and observation equations of the system are shown in (1).

$$\begin{cases} x_k = f(x_{k-1}, u_k, w_k) \\ y_k = h(x_k, v_k) \end{cases} \tag{1}$$

where x_k is the pose of the mobile robot at time k, u_k is the input of system motion equation at time k, w_k is the process noise of the system at time k, y_k is observation variables of the system at time k, v_k is observation noise of the system at time k. The problem is how to estimate the posterior probability of the system state from the observation variables with noise [8].

For Nonlinear-Non-Gaussian (NLNG) systems, the state estimation problem of the system can be solved well with particle filters. In the SLAM algorithm based on particle filters, the particle set is used to represent the probability distribution of the positioning result of the mobile robot. Each particle in the particle set represents the possible pose of the mobile robot [9]. The main steps of the algorithm include initialization, prediction, update, and resample. The specific implementation process is as follows:

1) Initialization

Initialize the sample set and obtain N particles $x_0^i, i = 1, 2, \cdots, N$ based on prior distribution $p(x_0)$. The initial weight of each particle is 1/N.

2) Prediction

According to the motion equation in (1), the prior probability at time k is predicted from the posterior probability at time k-1:

$$p(x_k \mid y_{1:k-1}) = \int p(x_k \mid x_{k-1}) p(x_{k-1} \mid y_{1:k-1}) dx_{k-1} \tag{2}$$

where $p(x_k \mid y_{1:k-1})$ is the prior probability at time k, $p(x_{k-1} \mid y_{1:k-1})$ is the posterior probability at time k-1.

3) Update

According to the observation equation in (1), the prior probability and observation data are used to obtain the posterior probability of state at time k:

$$p(x_k \mid y_{1:k}) = \frac{p(y_k \mid x_k)p(x_k \mid y_{1:k-1})}{\int p(y_k \mid x_k)p(x_k \mid y_{1:k-1})} \tag{3}$$

For the problem that the integral in (3) is difficult to calculate, the Monte Carlo sampling in the particle filter is used to approximate the calculation of the alternative posterior probability:

$$p(x_k \mid y_{1:k}) = \sum_{i=1}^{N_k} w_k^{(i)} \delta(x_k - x_k^{(i)}) \tag{4}$$

where $x_k^{(i)}$, $w_k^{(i)}$, N_k represent the state, weight, and number of particles at time k, $\delta(\cdot)$ is the Dirac Delta function [10]. Use the latest observation data, the update of the particle weights is calculated based on the likelihood function:

$$w_k^{(i)} \propto w_{k-1}^{(i)} \frac{p(y_k \mid x_k^{(i)})p(x_k^{(i)} \mid x_{k-1}^{(i)})}{q(x_k^{(i)} \mid x_{k-1}^{(i)}, y_k)} \tag{5}$$

4) Resample

Based on the principle of identically distributed, the re-sampling of the weighted updated particle set generates a new particle set and removes the particles with lower weights, so that most of the particle weights are almost equivalent, thus overcoming the problem of particle weight degradation.

Through the above steps, the robot can constantly estimate its own pose at each moment, and then draw an incremental two-dimensional grid map according to the obstacle information scanned by the Lidar.

III. Mobile Robot Platform

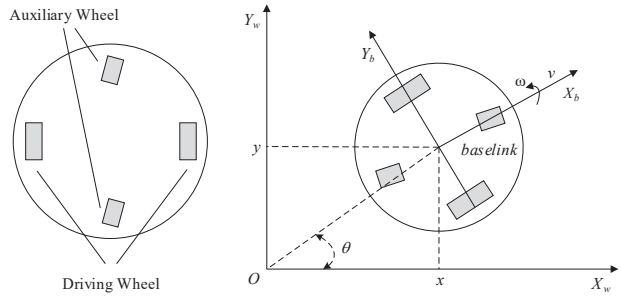
According to the task requirements of indoor mobile robots such as simultaneous localization and mapping, a flexible and reliable mobile platform that is easy to expand is designed based on the modular design concept, and the hardware system and software system are completed.

A. Mechanical Structure and Motion Model

The design of mobile robots uses a two-wheel differential drive. Differential drive is generally composed of two independent drive wheels. By adjusting the output power of the two drive motors, the mobile robot can be controlled to advance, retreat and rotate in place [11].

As shown in Fig. 1(a), the mobile robot chassis consists of two coaxial independent drive wheels and two coaxial auxiliary wheels. Each drive wheel is driven by a stepping motor. The speed of each drive wheel and the angle of rotation is controlled by inputting pulses of different frequencies. To ensure the stability of the mobile robot and the symmetry of the mechanical structure, two auxiliary wheels are also installed in the front and rear directions of the mobile robot to keep balance.

Before positioning, the pose of the mobile robot in space need to be characterized in a suitable way. The pose of a mobile robot consists of two parts. The first part is the location identification (x, y, z), which is the base link coordinate system of the mobile robot with respect to the world coordinate system. The second part is the gesture identification, which is the rotation transformation relationship between the mobile robot's base link coordinate system and the world coordinate system. Commonly used gesture characterization methods include Euler angle method, direction cosine method, quaternion method, and rotation vector method. In this paper, Euler angles are used to characterize the gesture of the mobile robot. That is, the yaw angle, pitch angle, and roll angle of the mobile robot are used to represent the angular position relationship of the base link coordinate system with respect to the world coordinate system. In three-dimensional space, the pose of a mobile robot can be expressed as $(x, y, z, \theta, \varphi, \phi)$. Considering that the mobile robot in the indoor environment is running on a flat ground, the pose of the mobile robot in the world coordinate system can be expressed as (x_w, y_w, θ) shown in Fig. 1(b).



(a) Chassis of mobile robot (b) Pose of the mobile robot

Fig. 1. Chassis and pose of mobile robot

In this paper, the dead reckoning is used to calculate the pose of the mobile robot at each moment. Using the velocity information of the mobile robot fed back by the sensor, the position and gesture of the robot relative to the world coordinate system can be calculated according to the equation, as shown in (6).

$$\begin{cases} x_{i+1} = x_i + v_i \cos\theta_i \Delta t \\ y_{i+1} = y_i + v_i \sin\theta_i \Delta t \\ \theta_{i+1} = \theta_i + \omega_i \Delta t \end{cases} \tag{6}$$

where v_i and ω_i represent the linear velocity and the angular velocity of the mobile robot at time k respectively.

B. Hardware System Design

The mobile robot mainly uses Acrylic material, which is lightweight and durable, as shown in Fig. 2.

The hardware system of the mobile robot is mainly composed of an upper controller, a bottom controller, a motor drive module, a power module, and sensors. The structure diagram of the mobile robot hardware system is shown in Fig.3. Among them, the STM32F103C8T6 processor based on the ARM Cortex-M3 architecture is used as the bottom layer controller. It is the core module of the underlying motion control system and is responsible for collecting the information of the underlying sensor, receive the control commands of the upper layer, and control the



Fig. 2. Physical map of mobile robot

speed of the motor at the same time. An industrial computer with high stability and low power consumption is used as the upper layer controller. It is the brain of the entire mobile robot and runs the robot operating system. The upper controller is responsible for some complex tasks of the robot, such as map construction, navigation and obstacle avoidance, image data processing, and communication with the host computer. The mobile robot uses stepper motor drive. Compared with the driving mode of the DC geared motor, the stepping motor not only has a simple mechanical structure and is easy to realize the control algorithm, but also has smaller accumulated error. The mobile robot sensor mainly uses Lidar and Microsoft's RGB-D camera Kinect, which can not only acquire the color image of the surrounding environment, but also directly obtain the depth information corresponding to the color image. Among them, the maximum resolution that the visual sensor can receive is 1280×960, infrared sensor can only support the image with maximum 640×480 resolution [12]. In addition, the IMU module is used to detect the angular velocity and acceleration of the mobile robot in three-dimensional space, and the data is used to correct the error of the odometer of the mobile robot.

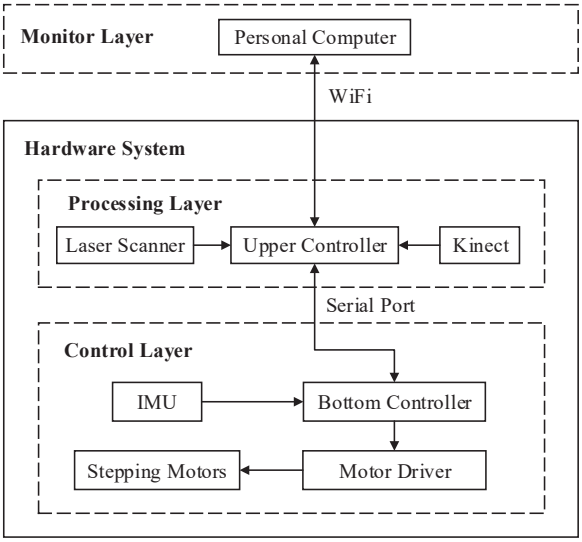


Fig. 3. Hardware system structure diagram of mobile robot

C. *Software System Design*

The robot's software system is designed based on the distributed framework of ROS. ROS originated from the Personal Robots Program, which was developed by Willow Garage and Stanford University's Artificial Intelligence Laboratory. After 2008, it was promoted and maintained by Willow Garage [13]. ROS provides functions similar to those provided by the operating system. It can provide users with functions such as hardware abstraction, monitoring the bottom layer of robots, information transfer between processes, and framework management of function packages.

The design goal of ROS is to realize the modularization of robot software and improve the portability and reuse rate of the code in the process of robot development. Therefore, ROS is designed as a distributed process framework, so that execution program can be designed independently and combined in real time. In addition, ROS also features point-to-point design, support for multiple programming languages, simplification and integration, ease of testing, application for large-scale program development, and open-source code, which greatly shortens the development cycle of robot software and reduces the development cost of robots [14]. When the program runs, all processes and their data processing will be represented by a peer-to-peer network form, which mainly include the following important concepts: master, nodes, message, topic and service. ROS takes a node as a unit to split a complex task into multiple independent modules, all of which register their own information to the node manager, and manage the message transfer and service calls between nodes by the node manager, as shown in Fig. 4.

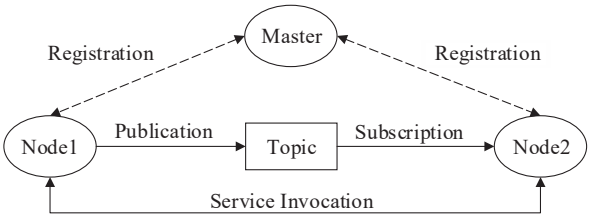


Fig. 4. Schematic diagram of ROS message delivery

The system software of the robot is divided into two parts, namely the robot body part and the upper computer part, which are connected together through the robot operating system. The body part of the mobile robot can be divided into two parts: the upper control system and the real-time control system, as shown in Fig. 5.

The operating system Ubuntu 16.04 installed with ROS runs on the host computer. The main software modules include RViz and keyboard control nodes. Among them, RViz is a 3D visualization software provided by ROS, which can be used to monitor the robot's movement status, displays sensor data, and releases navigation tasks to the robot. The keyboard control node is used to move the robot indoors to collect environmental information.

The Ubuntu 16.04 operating system runs in the upper control system of the mobile robot, and the robot operating system ROS is also installed. The ROS node manager runs in the upper layer control system, which is the core of the ROS network and is responsible for managing message delivery and task scheduling among the nodes. The Kinect driver controls the Kinect camera to capture environmental

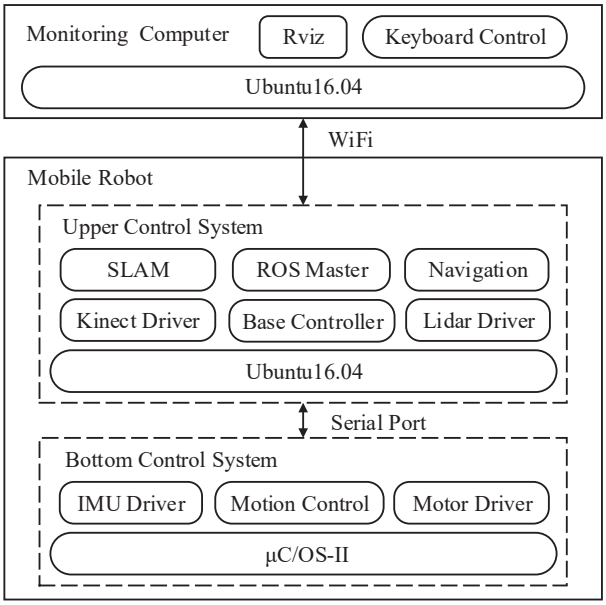


Fig. 5. Software system block diagram of mobile robot

information and publishes the acquired color image, depth image, and point cloud data to the corresponding topic. The radar driver controls the operation of the laser radar to collect environmental information and packages the laser scan data into a message. The SLAM node performs simultaneous positioning and map construction based on the sensor data collected by the robot and the calculated odometer data, and draws a map that is consistent with the surrounding environment. The navigation node designs a global path according to the constructed map and navigation task, and then determines its own position and the position of the surrounding obstacles based on the sensor information, and controls the movement of the robot toward the target point without collision with obstacles. The chassis controller node receives the speed control commands issued by other nodes, and then sends the speed commands to the underlying control system through the serial port, and combines the speed information fed back by the underlying control system with the IMU data to calculate the robot's pose.

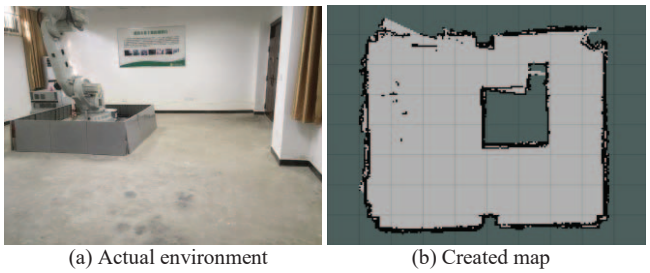
The real-time operating system μC/OS-II is transplanted in the bottom controller of the robot and is mainly responsible for sending the collected sensor data and the current speed of the robot to the upper-level control system in real time, and converting the speed control instructions of the upper-level control system into the frequency and number of pulses to control the rotation of the stepper motor.

IV. Experimental Results

To verify the feasibility of the designed scheme, map building experiment and autonomous navigation experiment were performed on mobile robots in indoor environment.

A. Map Building Experiment

Place the robot in an indoor environment as shown in Fig. 6(a). Launch the robot's Lidar, Kinect, and Chassis Controller nodes by remote login. Run the SLAM node on the host computer, control the robot to collect environment information around the room movement through the keyboard, view the incremental map constructed by RViz, and constantly adjust the robot's position and scanning angle until a complete map is constructed.



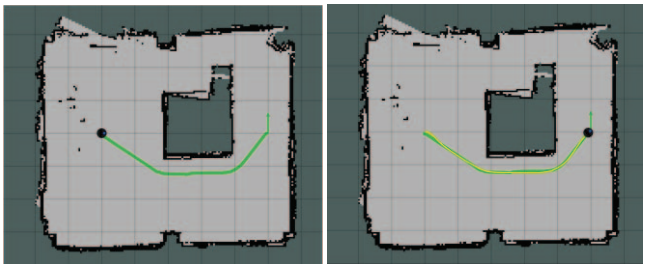
(a) Actual environment (b) Created map

Fig. 6. Created a 2D grid map

A grid map constructed by the mobile robot in the indoor environment is shown in Fig. 6(b), in which black areas represent obstacles such as walls, gray areas represent unknown spaces, and white areas represent freely movable spaces. Compared with the actual indoor environment and the constructed grid map, it can be seen that the mobile robot constructs a grid map basically consistent with the actual environment.

B. Autonomous Navigation Experiment

To verify the validity of the constructed map, an autonomous navigation experiment was performed on the robot with the constructed map. Load the constructed map into the navigation node and use the mouse to click on a location in the map in RViz as the target navigation point. The planned global path is shown in Fig. 7(a), and the moving trajectory of the mobile robot is shown in Fig. 7(b), where the green line segment is the planned global path and the yellow line segment is the actual motion path of the robot.



(a) Planned path (b) Trajectory of mobile robot

Fig. 7. Navigation path and robot motion trajectory

From Fig. 7(a) and Fig. 7(b), it can be seen that the mobile robot can plan an optimal path between the starting point and the ending point, and move toward the navigation target point according to the planned global path. During the movements of the robot, a partial path to avoid obstacles is planned according to the position of the surrounding obstacles, so that the robot's movement speed can be controlled in real time and eventually reach the navigation target point. It can be seen that the two-dimensional grid map constructed by the robot can be used for autonomous navigation of indoor mobile robots, which fully proves the effectiveness of the constructed map.

V. Conclusions

This paper designs a ROS-based indoor mobile robot map construction and autonomous navigation program. A low-cost, high-performance and easily-expanded mobile

robot hardware platform has been built. Using ROS's open source software framework for software development greatly reduces code repetitive writing and shortens the development cycle. The map building and autonomous navigation experiments were conducted in the actual indoor environment, which fully proved the feasibility of the scheme. Because the used laser radar is a single-line laser radar, only obstacles on the level of the laser radar can be detected. When the mobile robot autonomously navigates, obstacles on other levels cannot be effectively avoided. In the follow-up study, Kinect's image data can be used to make up for the inadequacy of single radar data, thereby improving the obstacle avoidance capabilities of mobile robots.

REFERENCES

[1] H. Durrant-Whyte, and T. Bailey, "Simultaneous Localization and Mapping: Part I," IEEE Robotics & Automation Magazine, vol. 13, pp. 99-100, June 2006.

[2] J. Bohren, R. B. Rusu, E. Gil Jones, E. Marder-Eppstein, C. Pantofaru, and M. Wise, "Towards autonomous robotic butlers: Lessons learned with the PR2," IEEE International Conference on Robotics and Automation, vol. 47, pp. 5568-5575, 2011.

[3] F. Tanaka, K. Isshiki, F. Takahashi, M. Uekusa, R. Sei, and K. Hayashi, "Pepper learns together with children: Development of an educational application," IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids), 2015, pp. 270-275.

[4] S. K. Agrawal, J. Kang, X. Chen, M. J. Kim, Y. Lee, and S. W. Kong, "Robot-Enhanced Mobility Training of Children With Cerebral Palsy: Short-Term and Long-Term Pilot Studies," IEEE Systems Journal, vol. 10, pp. 1098-1106, September 2016.

[5] Prado, José, and L. Marques, "Energy Efficient Area Coverage for an Autonomous Demining Robot," ROBOT2013: First Iberian Robotics Conference. Springer International Publishing, vol. 253, pp. 459-471, 2014.

[6] P. F. Zhang, K. Z. He, and Z. Z. Ouyang, "Multifunctional intelligent outdoor mobile robot testbed-THMR-V," Robot, vol. 24, pp. 97-101, 2002.

[7] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, and J. Leibs, "ROS: an open-source Robot Operating System," ICRA Workshop on Open Source Software, vol. 3, 2009.

[8] Wang, F. S., Ming-Yu, L. U., Zhao, Q. J., and Yuan, Z. J., "Particle Filtering Algorithm," Chinese Journal of Computers, vol. 37, pp. 1679-1694, 2014.

[9] Zhu, J. H., Zheng, N. N., Yuan, Z. J., Yong-Jian, H. E., Zhu, J. H., and Zheng, N. N., "A SLAM Approach by Combining ICP Algorithm and Particle Filter," Acta Automatica Sinica, vol. 35, pp. 1107-1113, 2009.

[10] LI Tian-Cheng, FAN Hong-Qi, SUN Shu-Dong, "Particle Filtering: Theory, Approach, and Application for Multitarget Tracking," Acta Automatica Sinica, vol. 41, pp. 1981-2002, 2015.

[11] C. Jung, and W. Chung, "Accurate calibration of two wheel differential mobile robots by using experimental heading errors," IEEE International Conference on Robotics and Automation, Saint Paul, MN, 2012, pp. 4533-4538.

[12] Zhang, Zhengyou, "Microsoft Kinect Sensor and Its Effect," IEEE MultiMedia, vol. 19, pp. 4-10, February 2012.

[13] Achmad, M. S. H., Murtdza, N. A., Lokman, N. A. A., Daud, M. R., Razali, S., and Pebrianti, D., "Exploration of Unknown Environment with Ackerman Mobile Robot using Robot Operating System (ROS)," Joint International Conferences on Electrical, Control and Computer Engineering, 2015, pp. 17573-17579.

[14] Fernandez, E., Crespo, L. S., Mahtani, A., and Martinez, A., Learning ROS for Robotics Programming - Second Edition, Packt Publishing, 2013.